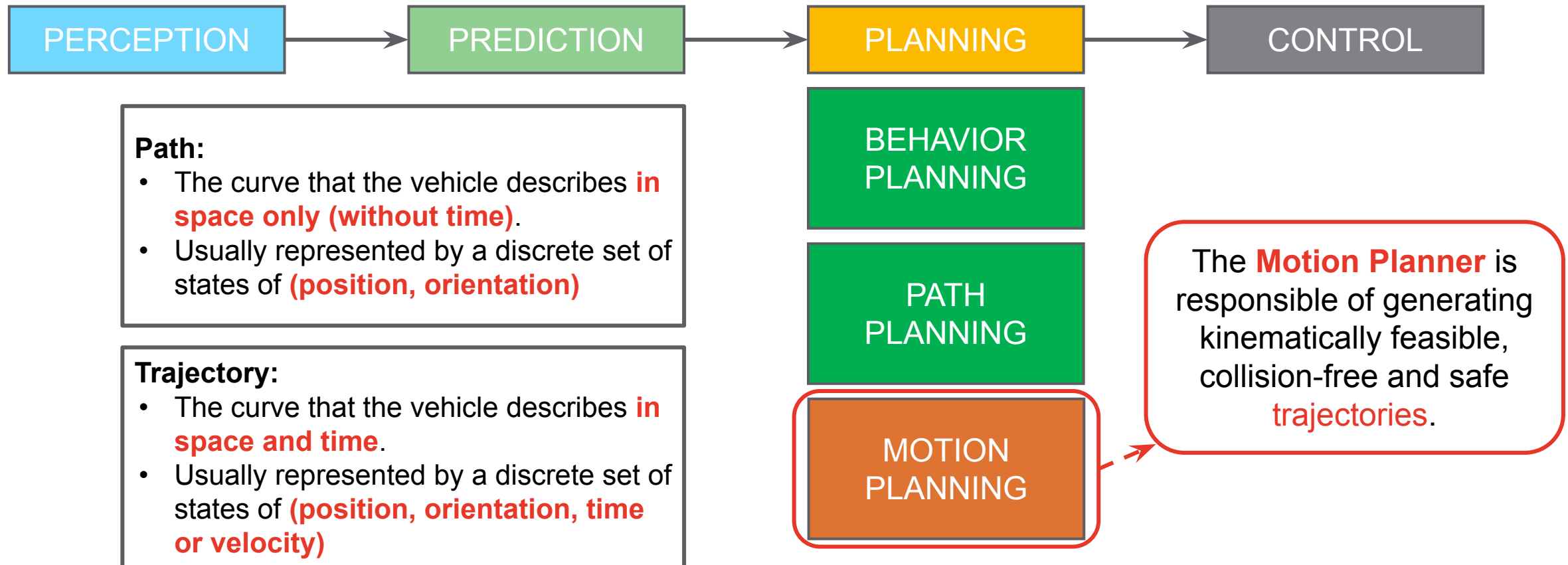


Motion Planning Problem Statement + Recap

Hierarchical Planning: Each Level can be described as its own optimization problem having a specific degree of abstraction with respect to the full problem.



Motion Planning Problem Statement + Recap

Given a starting configuration (eg. position, heading, curvature...), we want to find a trajectory to an goal configuration that:

1. Satisfies the vehicle's kinematic and curvature constraints
2. Avoids collision
3. Satisfies comfort and safety kinematic constraints

Direct Trajectory Generation

- Single Step (position time)
- **Minimum Jerk Trajectory** solver are an example.

Path Planning and Velocity Profile Generator

- Divide the problem into two:
 - Path Planning
 - Velocity Planning
- Take advantage of the multitude of approaches to solve each problem

Motion Planning Problem Statement + Recap

Direct Trajectory Generation

PROS

Closed-form solution

Computationally fast

Easy to implement

One step - directly generates trajectory

CONS

Need to estimate the time of the maneuver in advance

Curvature constraints are not included in the formulation (i.e NOT guaranteed)

Stuck with 1 single approach

Path Planning and Velocity Profile Generator

PROS

Flexibility and adaptability: Allows for multiple approaches depending on the driving scene (closer to real life!)

Can include multiple constraints into the formulation.

CONS

Computationally intensive: Requires Num Optimization, Num Approximation or Iterative approaches

More work to do and more code to develop

Path Planning

Path Planning

- Path is the **curve** that the vehicle describe in **space**
- we use a **parametric representation** to mathematically represent this curve: one or more **functions** of one or more independent parameters

Parametric Curve

- A **Parametric Curve “C”** in $\mathbb{R}^2$ is a set of equations (one for each dimension) $x = x(t)$ and $y = y(t)$ that trace a curve C as the **Parameter** t varies, usually from 0 to 1.
- Which means at $t = 0$, we will get X and Y at the “beginning” of the curve, and at $t = 1$, we will get X and Y at the end of the curve.

Splines

polynomial splines are **parametric curves** in which the equations that describe X and Y are **polynomials**.

Cubic spline example:

$$\begin{aligned}X(t) &= a_3t^3 + a_2t^2 + a_1t + a_0 \\Y(t) &= b_3t^3 + b_2t^2 + b_1t + b_0\end{aligned}$$

Path Planning

Spiral

A spiral, or more precisely a polynomial spiral is a parametric representation of a curve where instead of using equations that describe the position X and Y, we **use an equation that describe the “curvature” along the curve.**

spiral example:

$$k(s) = a_3s^3 + a_2s^2 + a_1s + a_0, \quad s = [0, s_f]$$

This is a **Cubic Spiral** since the polynomial used is of order 3, **k is the curvature as a function of the arc length s** (from 0 to the total length s_f).

Components of Path Planning

Path Planning Problem Statement

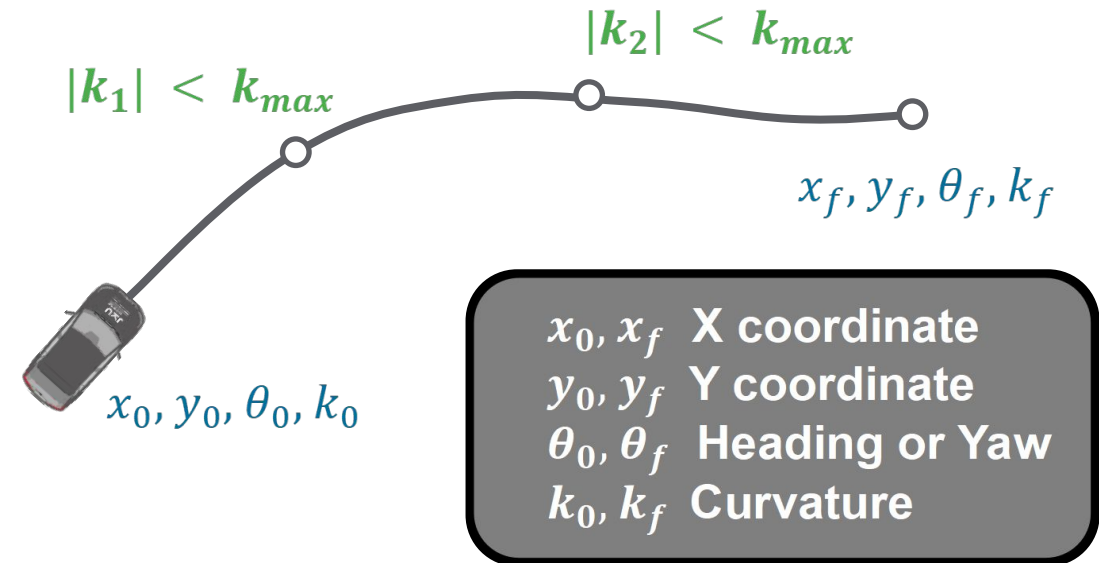
1. Find a path from a starting state to an ending state (position, heading and curvature)
2. Satisfy the vehicle's curvature constraints

Mathematically:

- Find the “coefficients” of a parametric curve that satisfies the above 2 conditions.

Components of the Path Planning Problem

1. **Boundary Conditions:** Specified by the start and end states.
2. **Constraints:** Curvature - k_{max} which is represented in the image on the right.
3. **Objective Function:** can be used to minimize the accumulated curvature (bending energy) along the path.



Path Planning: Polynomial Splines & Spirals

Using parametric curves in Path Planning: make setting up and solving the path planning problem way easier, which mainly means less computation loads and faster results.

Polynomial Splines: Are piecewise connected Parametric Curves where the equations that describe the coordinates X and Y are polynomials. One of the most commonly used Splines in Path Planning is the **Quintic Spline**, which have X and Y coordinates defined by 5th order polynomials:

$$\begin{aligned} X(t) &= a_5t^5 + a_4t^4 + a_3t^3 + a_2t^2 + a_1t + a_0 \\ Y(t) &= b_5t^5 + b_4t^4 + b_3t^3 + b_2t^2 + b_1t + b_0 \\ \text{where usually } t &= [0, 1] \end{aligned}$$

Polynomial Splines Challenges

Having $\mathbf{x}(s)$ and $\mathbf{y}(s)$ (s being a parameter) as the parametrization of the curve, **the curvature is defined as:**

$$k = \frac{(x' \cdot y'' - y' \cdot x'')}{(x'^2 + y'^2)^{\frac{3}{2}}}$$

Curvature constraints are hard to guarantee due to:

- **Discontinuities on $\kappa(s)$**
- **Discontinuities on its derivative: $\kappa'(s)$.**

Path Planning: Polynomial Splines & Spirals

Polynomial Spirals

Curves whose **curvature is a polynomial function of the arc length s** are called polynomial spirals

One of the most commonly used Polynomial Spirals in Path Planning is the Cubic Spiral

$$k(s) = a_3 s^3 + a_2 s^2 + a_1 s + a_0, \quad s = [0, s_f]$$

Polynomial Spirals Challenges

Having the curvature as the parametrization of the curve, it turns out that to get the coordinates X, Y for every point we need to solve two integrals that have no closed-form solution.

$$\theta(s) = \theta_0 + \int_0^s k(s') ds'$$

$$\theta(s) = \theta_0 + \int_0^s (a_3 s'^3 + a_2 s'^2 + a_1 s' + a_0) ds'$$

$$\theta(s) = \theta_0 + \frac{a_3 s^4}{4} + \frac{a_2 s^3}{3} + \frac{a_1 s^2}{2} + a_0 s$$

$$x(s) = x_0 + \int_0^s \cos(\theta(s')) ds'$$

$$y(s) = y_0 + \int_0^s \sin(\theta(s')) ds'$$

No closed-form solution

To solve this problem we use **Simpson's Rule** to Approximate the solution for both.

Simpson's Rule

$$\int_a^b f(x) dx \approx \frac{\Delta x}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \dots + 4f(x_{n-1}) + f(x_n))$$

$$\text{where } \Delta x = \frac{b-a}{n} \text{ and } x_i = a + i\Delta x$$

Exercise: Simpson's Rule

Simpson's Rule

In this exercise we will calculate the value of the following integral:

$$x(s) = x_0 + \int_0^s \cos(\theta(s')) ds' \quad \text{NOTE: } x_0 \text{ will be 0 in this exercise.}$$

To do so and since this integral does NOT have a closed-form solution we will use Simpson's Rule.
The only 2 things you need are:

- The definition of Theta(s) for a Cubic Spiral, which is:

$$\theta(s) = \theta_0 + \frac{a_3 s^4}{4} + \frac{a_2 s^3}{3} + \frac{a_1 s^2}{2} + a_0 s \quad \text{NOTE: } \theta_0 \text{ will be 0 in this exercise.}$$

- Simpson's Rule equation, which is:

$$\int_a^b f(x) dx \approx \frac{\Delta x}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \dots + 4f(x_{n-1}) + f(x_n))$$

where $\Delta x = \frac{b-a}{n}$ and $x_i = a + i\Delta x$

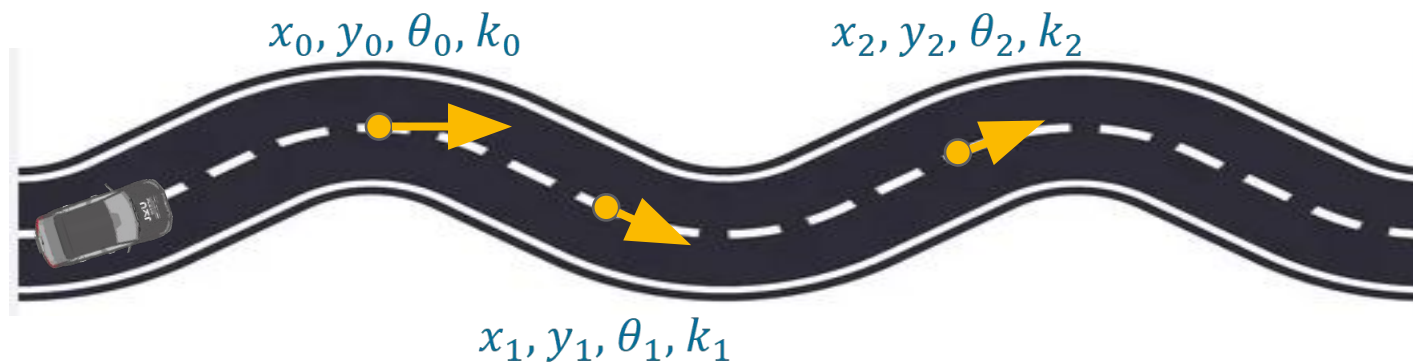
Download code from here: EX Planning-20230423T175220Z-001.zip in Moodle, part 3
Follow the "TODO"s to complete this exercise.

Boundary Conditions

Goal Location

In order to find ideal goal position, heading, and curvature - resource the road structure by:

- Locating the goal in the centerline of the target lane
- Heading in the tangential direction of the road
- Having the same curvature of the road at that point.



Boundary Conditions

Goal Distance Calculation

To calculate the goal distance, you need to base it on:

- Ego velocity
- Distance and speed of any vehicle ahead
- Environmental factors, such as road conditions and weather

There is a trade-off to be made in terms of how far we place the goal that could be affected whether it's too close or too far.

Too close	
Pro	Con
Computationally fast	Short sighted - lose ability to plan smooth obstacle free paths.
Too far	
Pro	Con
Ability to plan smooth obstacle free paths.	Computationally too intensive - Might not be suitable for real-time applications.

Boundary Conditions

Offset Goals

are necessary alternatives in case the center goal cannot be achieved

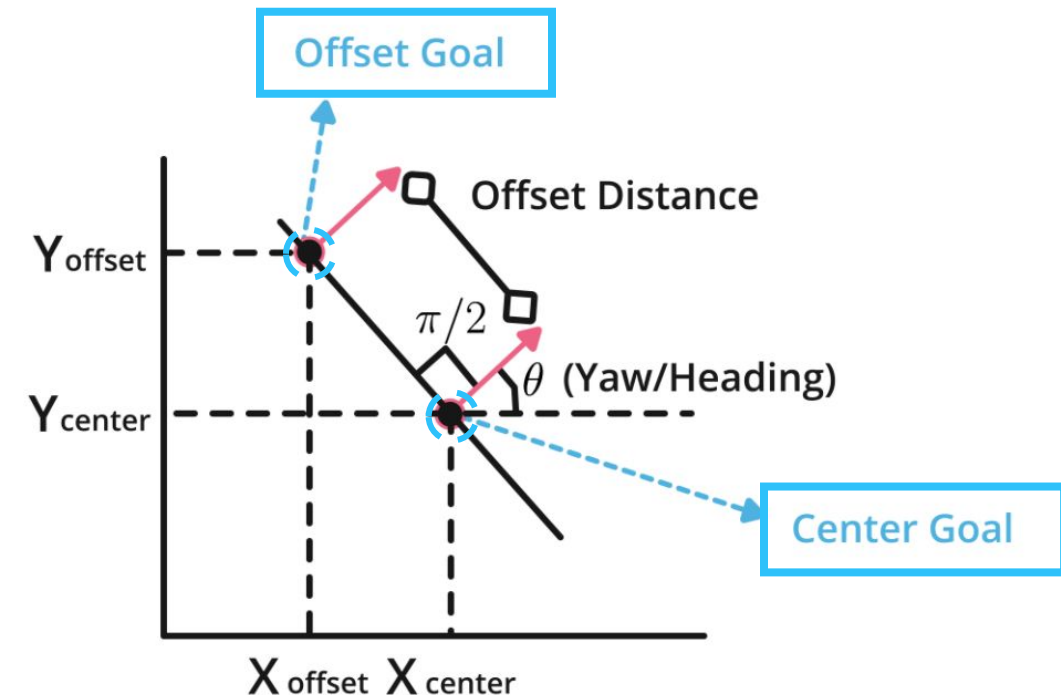
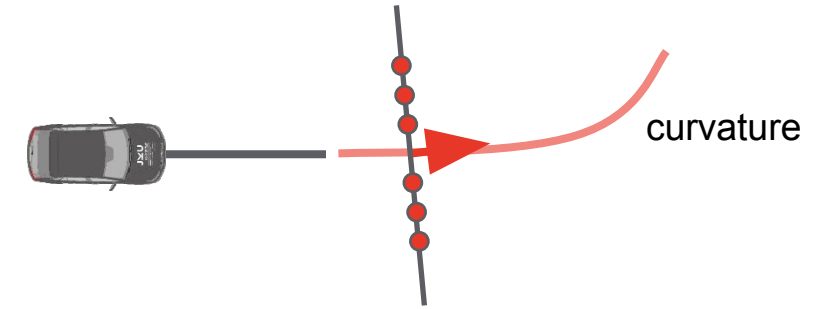
- Offset by a constant distance on each side of the center goal.
- Lying on a perpendicular line to the center goal heading.
- Heading on the same direction of the center goal and with the same curvature.

To calculate the offset goals location and heading:

$$X_{\text{offset}} = X_{\text{center}} + \text{goal_number} * \text{Offset_distance} * \cos(\theta + \pi/2)$$

$$Y_{\text{offset}} = Y_{\text{center}} + \text{goal_number} * \text{Offset_distance} * \sin(\theta + \pi/2)$$

goal_number = [-n, ..., 0, ...n] (Eg: for 5 total goals [-2, -1, 0, 1, 2])



Exercise: Boundary Conditions: Offset Goals

Boundary Conditions: Offset Goals

In this exercise you will generate "_num_goals" goals, offset from the given center-goal at a distance "_goal_offset". The offset goals will be aligned on a perpendicular line to the heading of the main goal. To get a perpendicular angle, just add 90 degrees (or $\pi/2$ rad) to the main goal heading (Θ). After that you will just need to calculate the x and y coordinates for each offset goal using the equations presented in the lectures:

$$X_{\text{offset}} = X_{\text{center}} + \text{goal_number} * \text{Offset_distance} * \cos(\theta + \pi/2)$$

$$Y_{\text{offset}} = Y_{\text{center}} + \text{goal_number} * \text{Offset_distance} * \sin(\theta + \pi/2)$$

NOTE:

goal_number will go from: -3, -2, -1, 0, 1, 2, 3 (for 7 total goals)

When goal_number = 0, we will get the "center" goal.

Download code from here: EX Planning-20230423T175220Z-001.zip in Moodle part 3

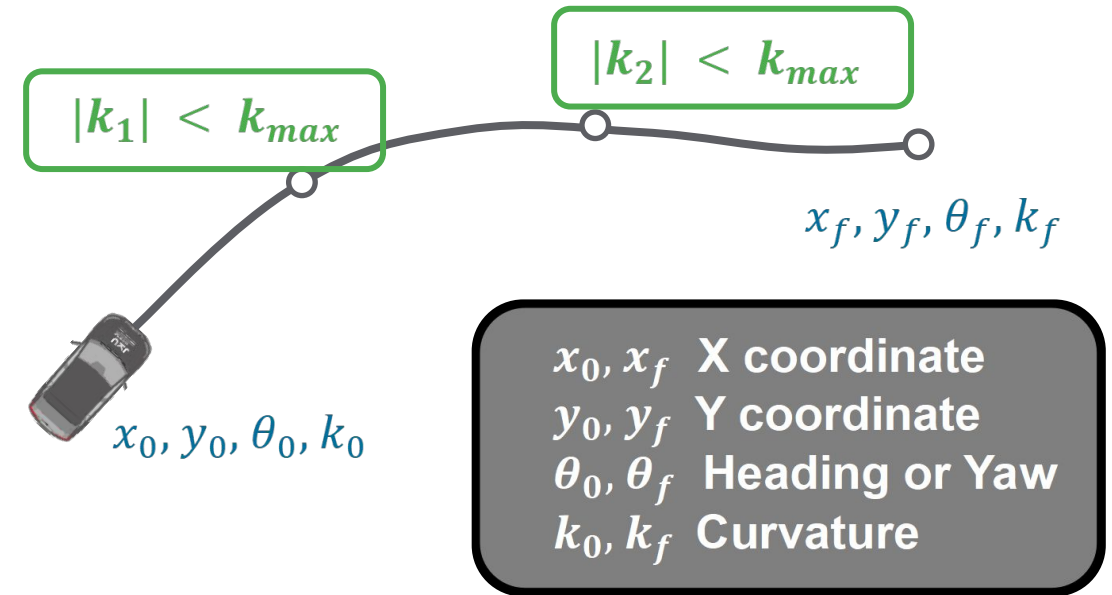
Follow the "TODO"s to complete this exercise.

Constraints: Curvature

A Curvature Constraint

is the minimum turning radius of a vehicle

- We want to apply this curvature constraint to the path so it is drivable
- We can not apply this constraint to infinite points on the path, so we sample a few points.
- Fewer constraints, easier problem
- It's common across literature to use 2 constraints, at 1/3rd and 2/3rd of the way respectively.



Numerical Approximation vs. Numerical Optimization

Finding the "best" coefficients for our polynomial spiral requires either:

- **Numerical Approximation:** Techniques that give approximate but accurate solutions to problems, or
- **Numerical Optimization:** Minimizing or maximizing a function by selecting a sample of elements from the large complete distribution of possibilities.

What do we need to solve the path planning problem?
for Both:

- **Boundary Conditions:** start and end states
- **Constraints:** curvature - k_{max}

for Numerical Optimization only:

- **Objective Functions:** a function to minimize

Both Approaches' Challenge: Satisfy "End State" exactly

- Numerical Approximation: Setting up a "Tolerance"
if (error < tolerance)
success!
- Numerical Optimization: Including terms into the objective function that penalize deviating from the state

$$ObjFunc = f(k) + a(X(sf) - Xf) + \beta(Y(sf) - Yf) + \gamma(\theta(sf) - \theta f)$$

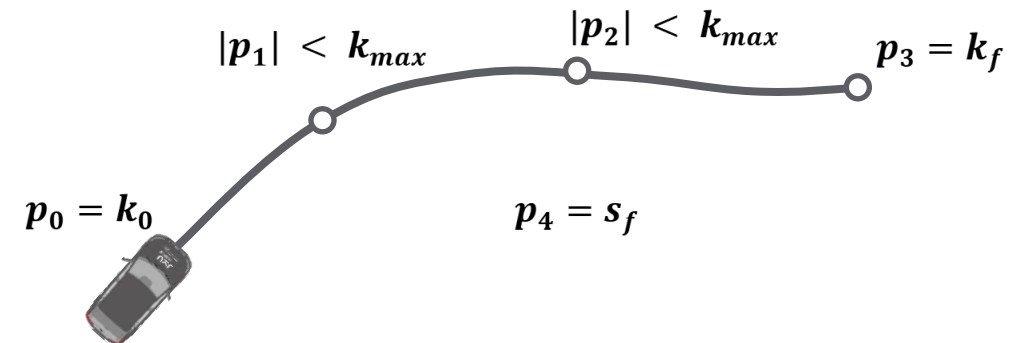
Numerical Approximation vs. Numerical Optimization

Simplifying the problem: Parameter mapping

- Curvature parameterization has 4 coefficients (3 unknowns): a_3, a_2, a_1, a_0
- We also do not know s_f = the total arc length of the spiral. (i.e. 4 unknowns for now)
- We introduce a set of **5 parameters “p”**:
 - p_0, p_1, p_2, p_3 = curvature at 4 points evenly spaced along the spiral
 - p_4 is the spiral length

- p_0 and p_3 are known: Initial and final curvature
- We are left with 3 unknowns
- We reduce the dimensionality from 4 to 3, so we just need to calculate p_1, p_2, p_4

$$k(s) = a_3 s^3 + a_2 s^2 + a_1 s + a_0$$
$$k(0) = a_0 = k_0$$



Discretization Resolution

Importance of Discretization Resolution

Solving the Path Planning problem using numerical approximation or optimization, we obtain a spiral that represents a continuous curve for infinite values of s , from 0 to s_f :

$k(s) = a_3s^3 + a_2s^2 + a_1s + a_0$ where all “a” parameters are now known.

We still need to **discretize** this continuous curve, so it can be feasibly and efficiently used for the following tasks: **Collision Checking**; **Velocity profile generation**; **Motion control**.

How fine or coarse should the discretization be?



Discretization Resolution

Pros		Cons	
Too coarse			
fast - we only perform collision checking, assign velocities and generate control commands for a few points.		- Inaccurate collision check - we might even miss a collision! - Jerky and undesirable vehicle motion - there may not be enough points for the motion controller to be smooth.	
Too fine			
Accurate collision detection Smooth motion controller		Computationally too intensive - potentially not suitable for real-time planning.	

Finding a Good Discretization Resolution

Ideally, this resolution should be calculated dynamically based on:

- Curve arc length (s_f)
- Curvature changes $[\kappa'(s)]$
- We might also want to add more points when making tighter curves

For simplicity and convenience, in many cases we always plan a constant distance ahead, then we can use a constant number of points to discretize the curve. So that we have enough points to account for tight curves.

Velocity Profile Generation

Velocity Profile

We need to add a time element to our state vector $(x_i, y_i, \theta_i, k_i)$ by either:

- Calculating the “time” at which the vehicle needs to reach every waypoint
- Assigning a speed to each waypoint

We will choose the second one since it will give us a nice and convenient target (the speed) for the motion controller.

Velocity Profile Generation Steps:

1. Calculate the Final Speed (V_f) at the end of the path
2. Follow a speed profile (adequate for the driving scene) to assign speed values to all path points in between.

Velocity Profile Generation

Calculate the Final Speed

What do we need to take into account to calculate the final speed V_f ?

- The Behavioral Planner reference speed: V_{ref}
 - Path without stop signs or red lights: V_{ref} = road speed limit
 - Stop maneuver (red light, stop sign): V_{ref} = 0 m/s
- Leading vehicle speed: V_{lead}
- Speed based on the curvature at the goal and the max comfortable lateral acceleration allowed: V_{kf}

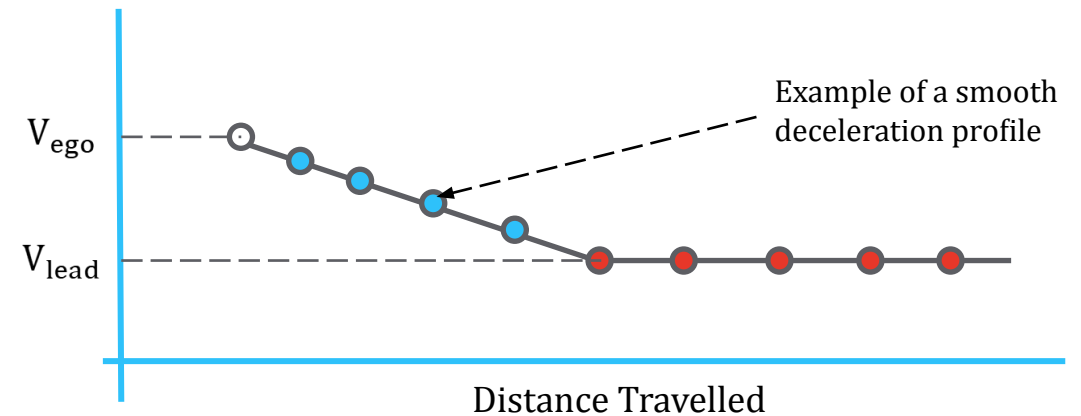
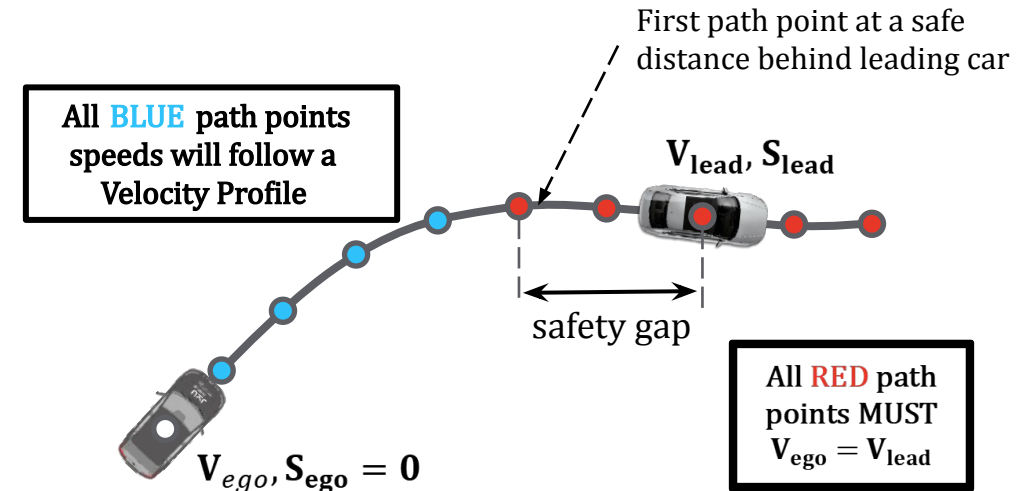
The Final Speed will be:

$$V_f = \min(V_{ref}, V_{lead}, V_{kf})$$

Velocity Profile Generation

Example: Velocity Profile when we have a vehicle ahead of us

- The first waypoint **behind** a safety distance from the vehicle ahead **MUST** have a speed not higher than V_{lead}
- The rest of the waypoints **ahead this point** should **keep** that speed (V_{lead}) to make sure there is no collision.
- A Velocity Profile will be applied **only** to the waypoints **behind**. This profile will depend on the driving scene. We might want to accelerate or we might have been traveling faster so we will slow down to V_{lead} .



Velocity Profile Generation

Velocity limit based on the path point curvature (k_i) and max comfortable lateral acceleration allowed

$$V_{ki} \leq \sqrt{\frac{a_{maxLat}}{k_i}}$$

In this case, we are only interested in the speed limit at the final point with curvature (k_f): V_{kf}

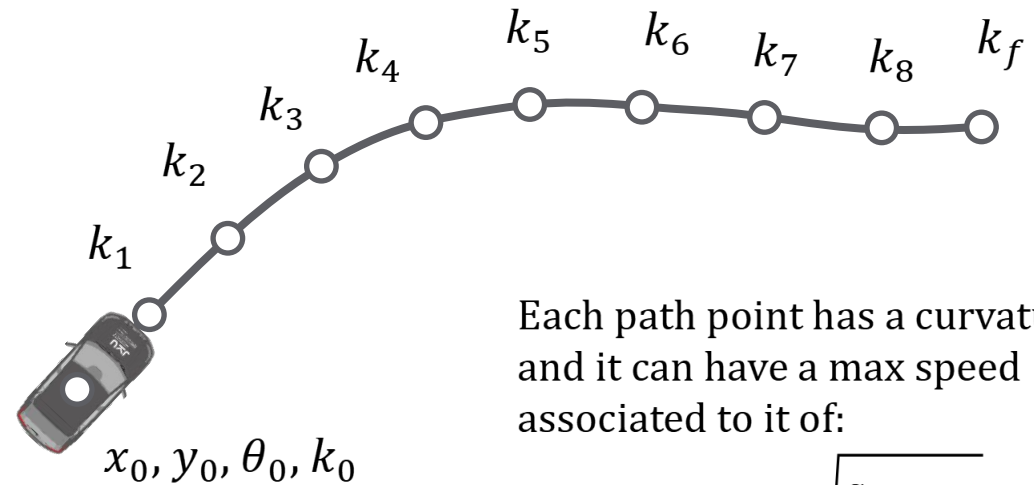
Velocity assignment for the final point

$$V_f = \min(V_{ref}, V_{lead}, V_{kf})$$

Behavioral Planner
Reference Speed

Leading
Vehicle Speed

Curvature
Based Speed



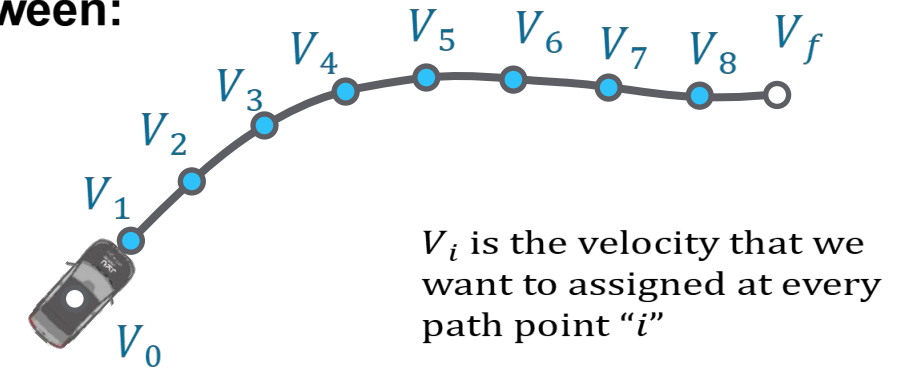
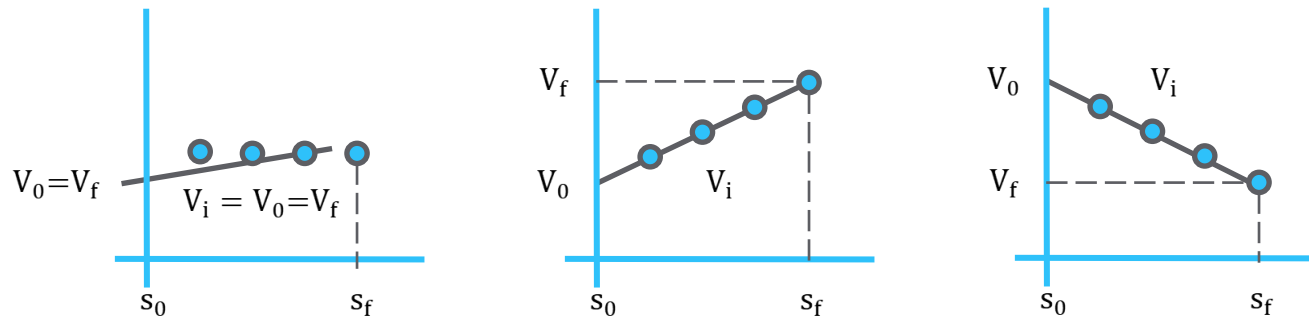
Each path point has a curvature k_i and it can have a max speed associated to it of:

$$V_{ki} \leq \sqrt{\frac{a_{maxLat}}{k_i}}$$

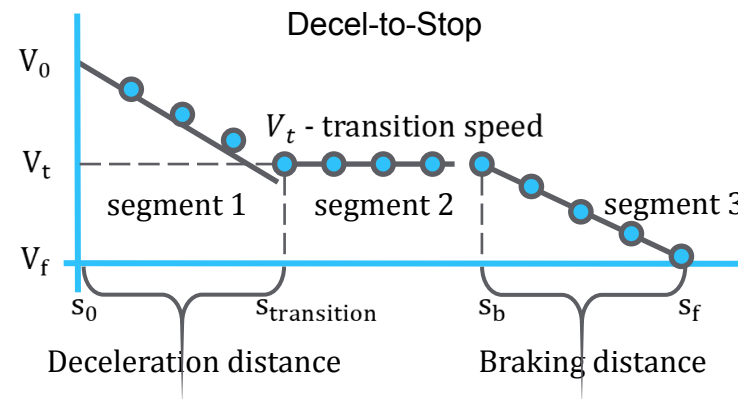
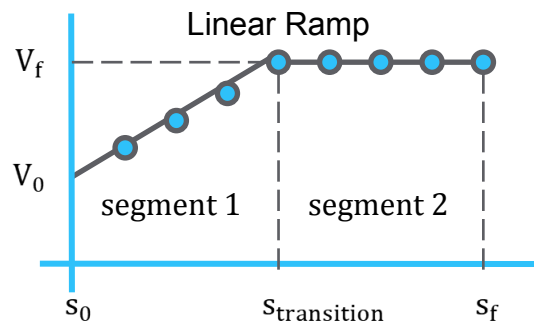
Velocity Profile Generation

Now we need to decide how we would generate all the speeds between:
 V_0 and $V_f : V_i$ ($i = 1, 2, 3, \dots$)

- Linear profiles: Constant Acceleration



- Linear composition profiles: Linear Ramp & Decel-to-Stop profiles (Trapezoidal)



Horizontal axes: Distance travelled along the curve
Vertical axes: Ego's velocity

Velocity Profile Generation: Linear Velocity Profiles

1. Constant Acceleration:

$$a = \frac{(V_f^2 - V_0^2)}{2s_f}$$

s_f is known from solving the Path Planning Problem and is the total length of the curve

2. Every single path point speed:

$$V_i = \sqrt{2as_i + V_0^2}$$

s_i – distance travelled from origin to point “i”

3. Velocity limit due to curvature:

$$V_{ki} \leq \sqrt{a_{\max lat}/k_i}$$

k_i – curvature at point “i”

4. The final speed of each point:

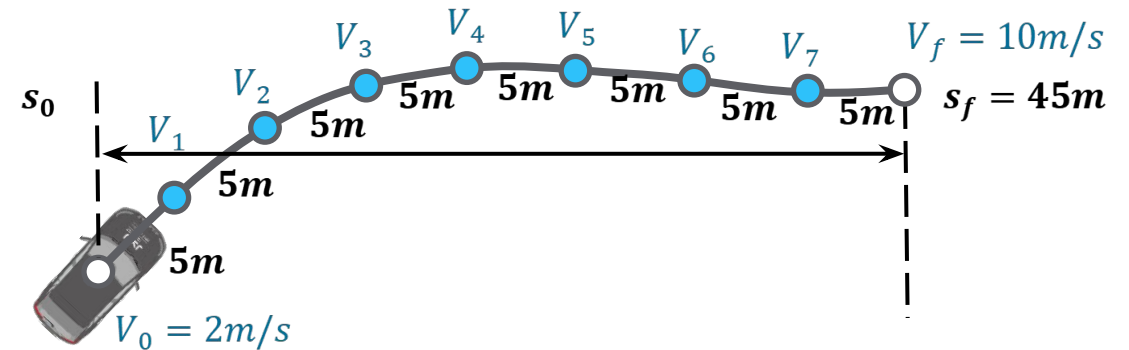
$$V_{if} = \min(V_i, V_{ki})$$

Example:

Initial velocity $V_0 = 2\text{m/s}$

Final velocity $V_f = 10\text{m/s}$

Distance $s_f = 40\text{m}$



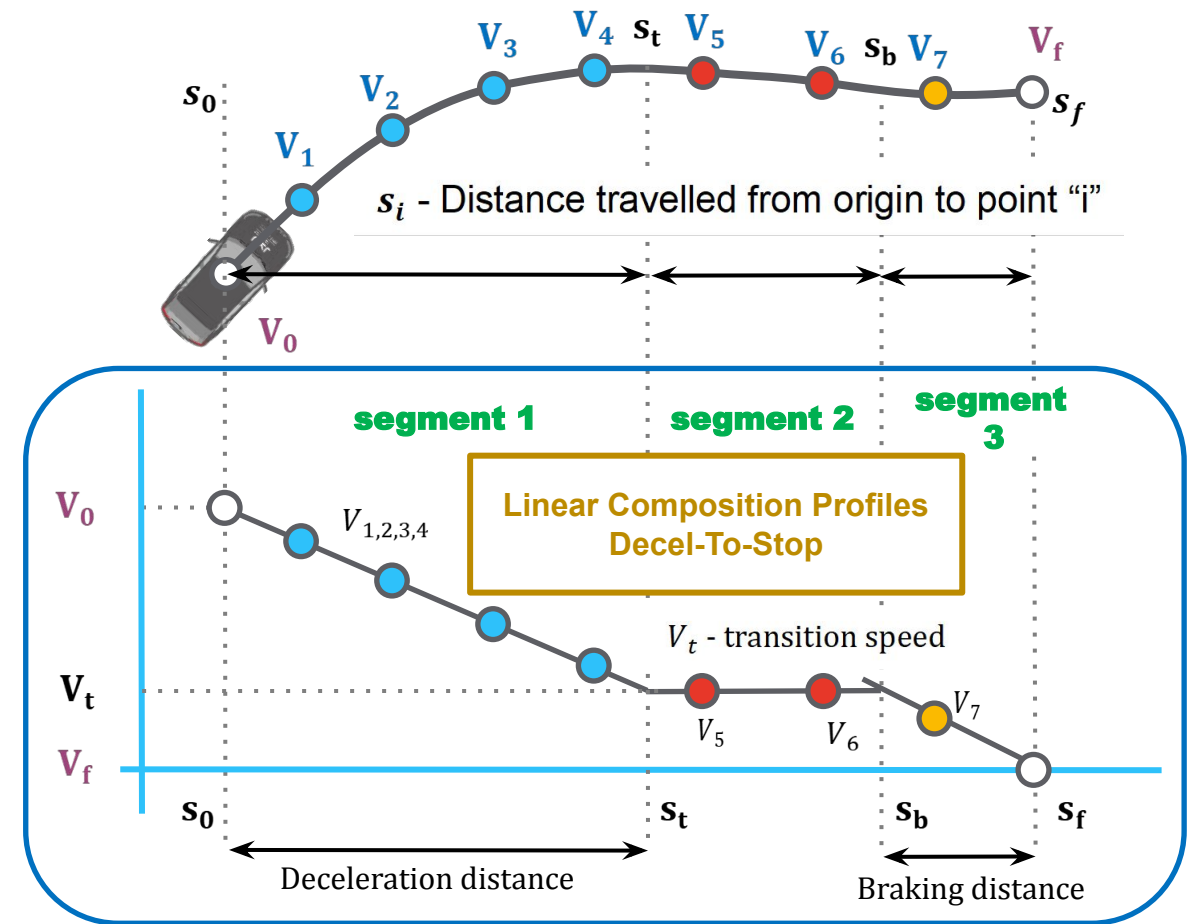
$$1. \quad a = \frac{(10^2 - 2^2)}{2 \cdot 40} = \frac{96}{80} = 1.2\text{m/s}^2$$

$$2. \quad V_i = \sqrt{2as_i + V_0^2} \quad \text{where: } i = 1, 2, \dots, 7; \quad s_i = 5, 10, \dots, 35$$
$$V_1 = \sqrt{2 \cdot 1.2 \cdot 5 + 2^2} = 2.82843$$
$$\vdots$$
$$V_7 = \sqrt{2 \cdot 1.2 \cdot 35 + 2^2} = 8.94427$$

Velocity Profile Generation: Linear Composition Profiles

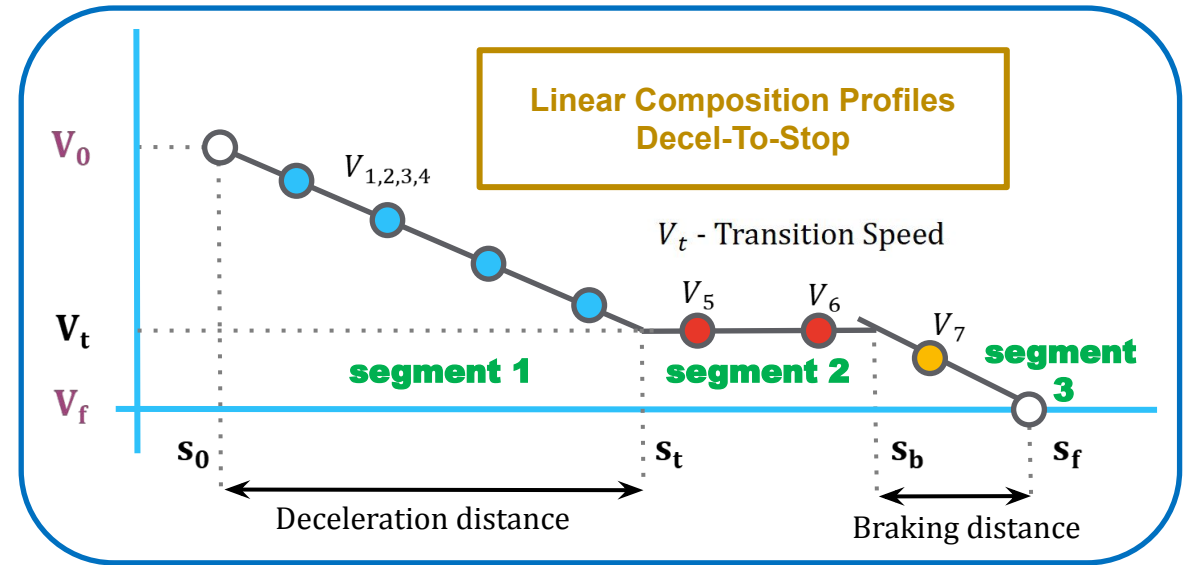
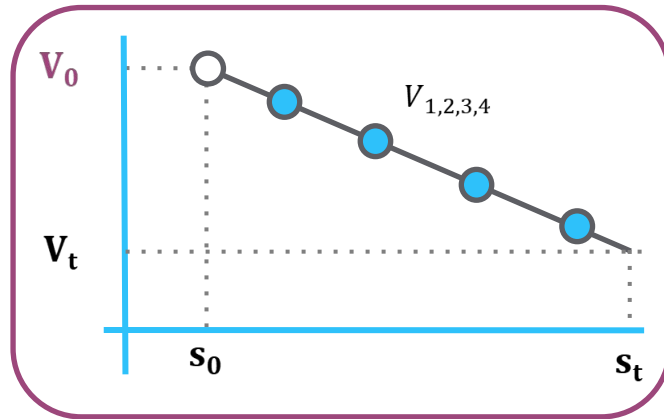
Decel-to-Stop Profile Example:

- Including three linear segments (see the right image)
- Segment 1: constant deceleration segment. We can get s_t by using V_0 , V_t and a gentle deceleration a . Then we can calculate V_i for all points in this segment.
- **Segment 3:** to get s_b , we need to use the final speed V_f , the start speed of 3rd segment V_t , and a gentle deceleration a . With s_b , we can calculate V_i for all points in this segment.
- Segment 2: constant speed segment. all $V_i = V_t$
- Note that both V_t and a are designed parameters.



Velocity Profile Generation: Linear Composition Profiles

Segment 1:



1st Step: Calculate the distance s_t required to reach V_t from V_0 using a gentle accel/deccel a . V_t and a are design parameters

$$s_t = \frac{(V_t^2 - V_0^2)}{2a}$$

2nd Step: Calculate the speed for every point that falls between s_0 and s_t

$$V_i = \sqrt{2as_i + V_0^2}$$

si - distance travelled from origin to point "i"

3rd Step: Calculate the speed of these same points based on their curvature (k_i) and the max comfortable lateral acceleration allowed

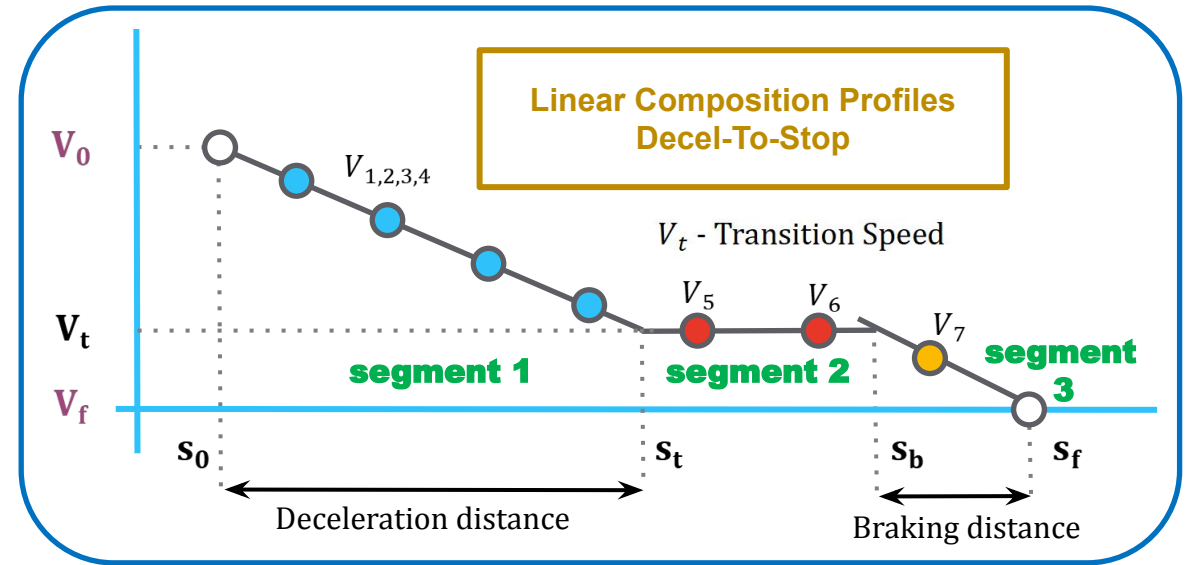
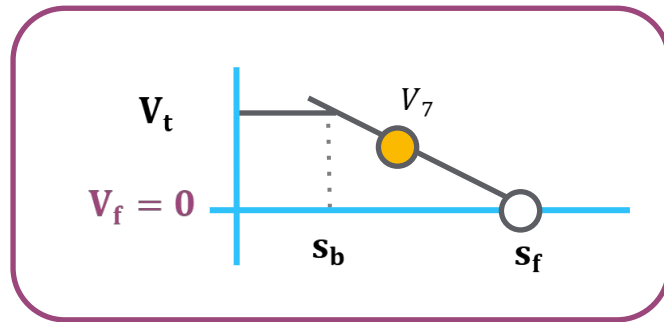
$$V_{ki} \leq \sqrt{\frac{a_{maxlat}}{k_i}}$$

Final Step: Chose the minimum

$$V_{if} = \min(V_i, V_{ki})$$

Velocity Profile Generation: Linear Composition Profiles

Segment 3:



1st Step: Calculate the distance required to reach V_f from V_t using a gentle accel/deccel a . Recall that a is design parameter

$$s_b = s_f - \frac{(V_f^2 - V_t^2)}{2a}$$

2nd Step: Calculate the speed for every point that falls between s_b and s_f

$$V_i = \sqrt{2a(s_i - s_b) + V_0^2}$$

s_i - distance travelled from origin to point "i"

3rd Step: Calculate the speed of these same points based on their curvature (k_i) and the max comfortable lateral acceleration allowed

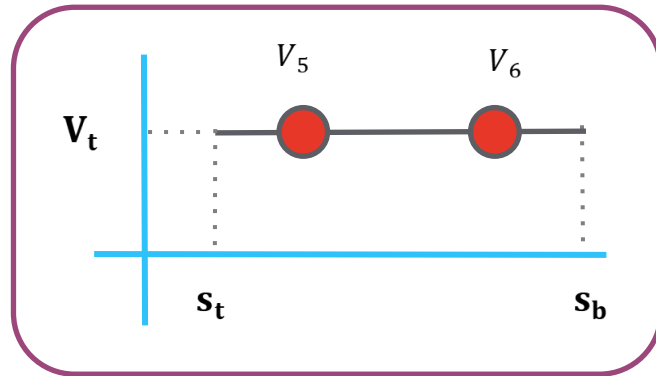
$$V_{ki} \leq \sqrt{\frac{a_{maxlat}}{k_i}}$$

Final Step: Chose the minimum

$$V_{if} = \min(V_i, V_{ki})$$

Velocity Profile Generation: Linear Composition Profiles

Segment 2:

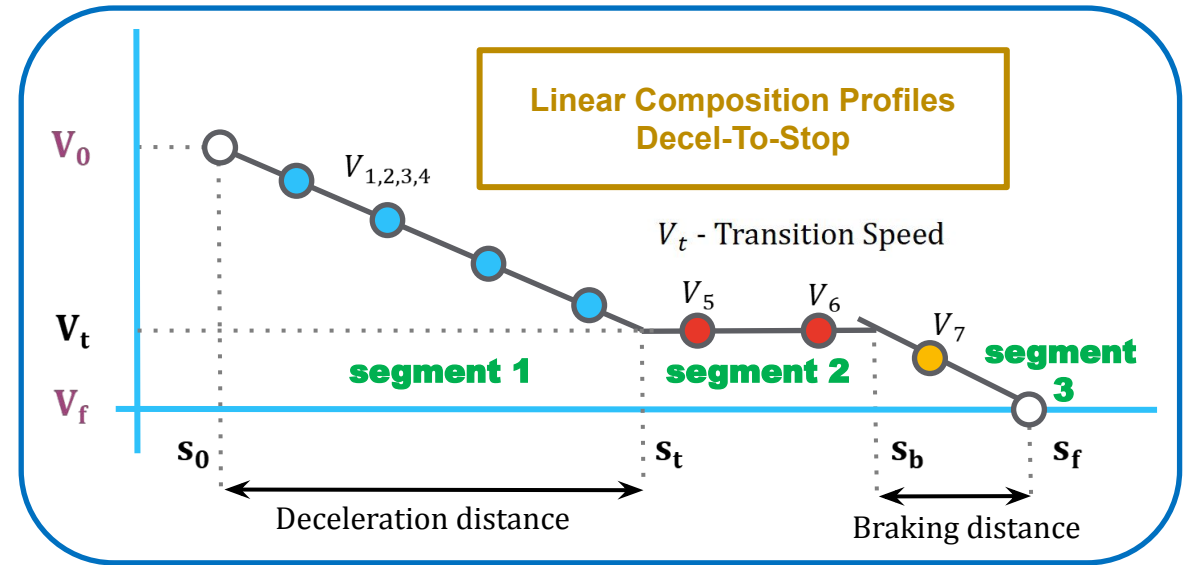


1st Step: All points that fall between s_t and s_b will be assigned a constant speed V_t

$$V_i = V_t$$

2nd Step: Calculate the speed of all 2nd segment points based on their curvature (k_i) and the max comfortable lateral acceleration allowed

$$V_{ki} \leq \sqrt{\frac{a_{maxlat}}{k_i}}$$

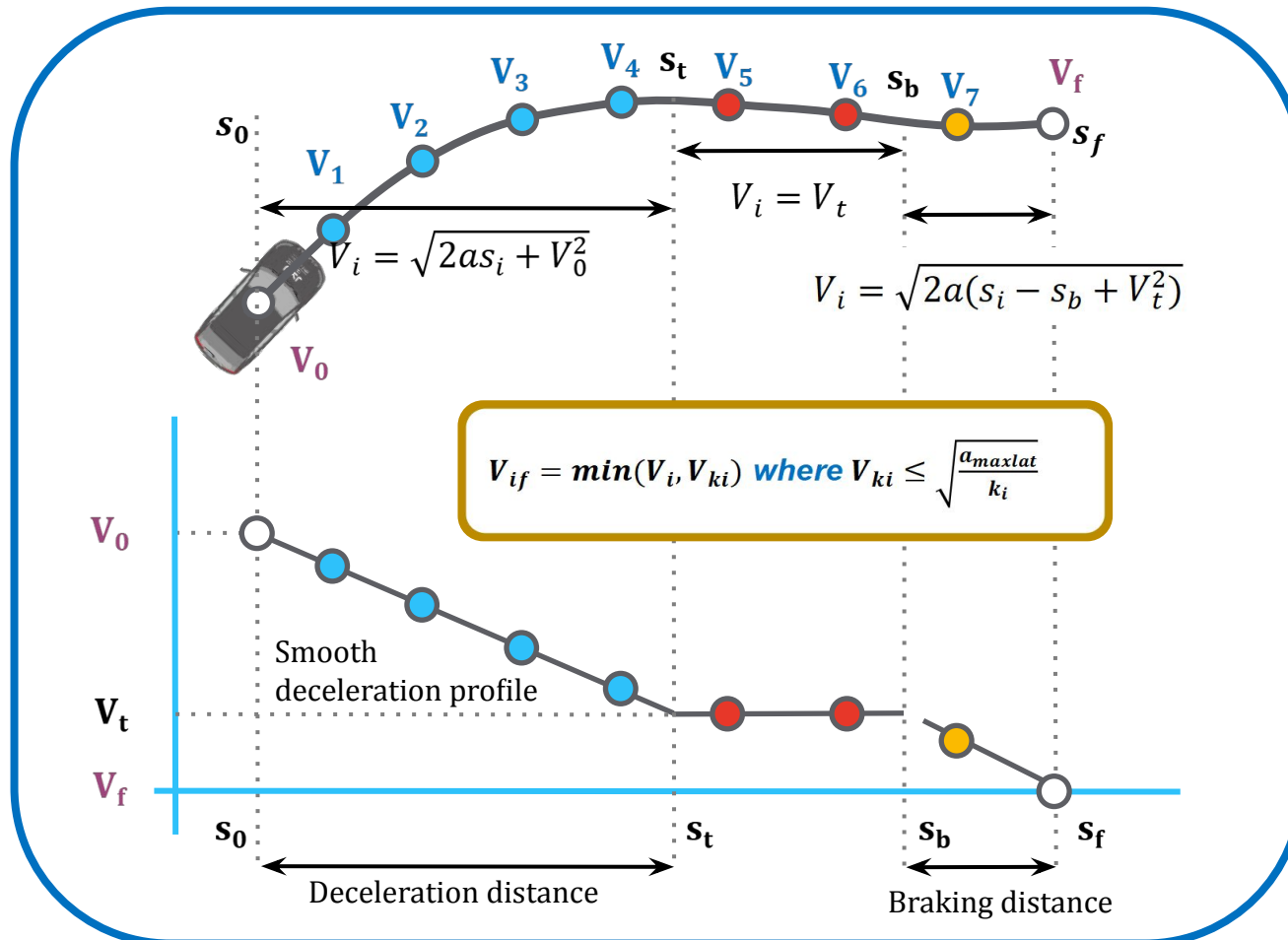


Final Step: Chose the minimum

$$V_{if} = \min(V_i, V_{ki})$$

Velocity Profile Generation: Linear Composition Profiles

Summary:



NOTE

s_i - Distance travelled from origin to point "i"

k_i - Curvature at point "i"

V_t - Transition Speed. Profile parameter we chose

a - Smooth Accel/Decel. Profile parameter we chose

Exercise: Velocity Profile Generation

Problem Statement

In this exercise you will practice 2 essential calculations necessary to build a linear profile.

calculate the a final speed (v_f) given an initial speed (v_i), an acceleration (a) and distance (d).

$$v_f = \sqrt{v_i^2 + 2ad};$$

calculate the distance traveled (d) given an initial and final speeds (v_i & v_f) and acceleration (a).

$$d = (v_f^2 - v_i^2) / (2 * a);$$

Download code from here: EX Planning-20230423T175220Z-001.zip in Moodle part 3

Follow the "TODO"s to complete this exercise.

Collision Detection

Characteristics of Collision Detection:

Due to its safety-critical nature, collision detection is one of the most important and challenging tasks in motion planning.

We will focus on the most relevant characteristics of collision detection for autonomous vehicles:

- The real-time requirements of collision detection
- The safety-critical nature of collision detection

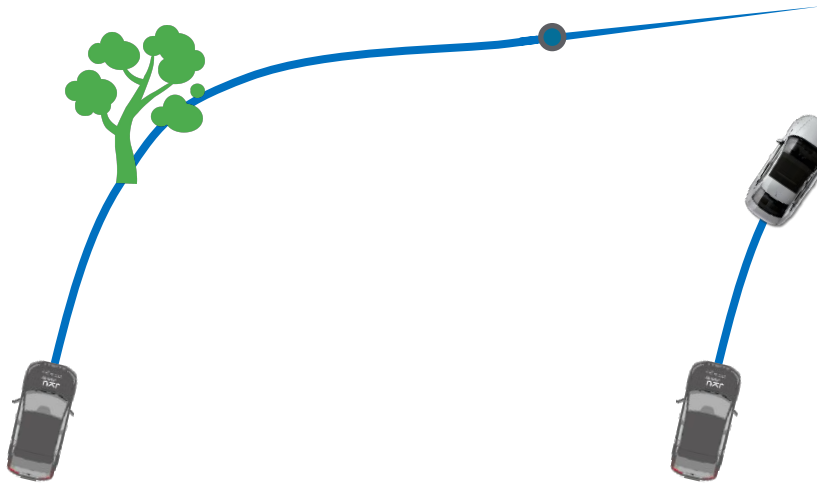
Types of Collision Detection:

- **Dynamic collision detection** tries to detect collisions with objects that are in motion.
- **Static collision detection** detects any static objects or obstacles that are on ego's car path.

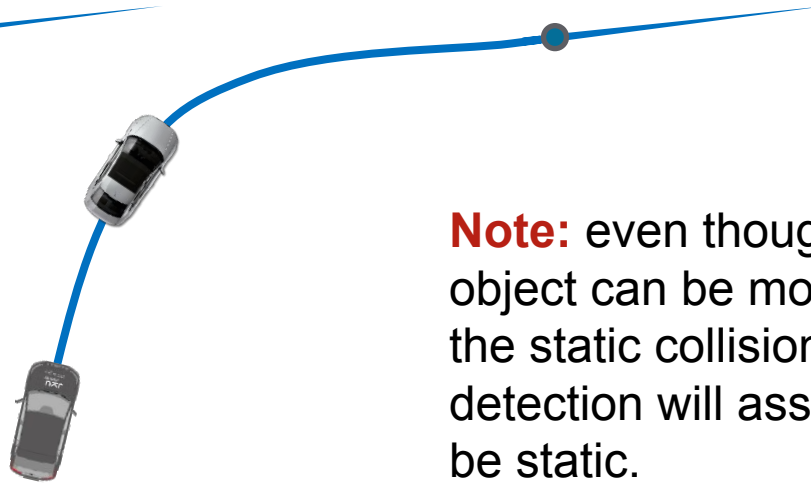
Collision Detection

Static Collision Detection

In static collision checking, objects are assumed to be static even though they might be moving.

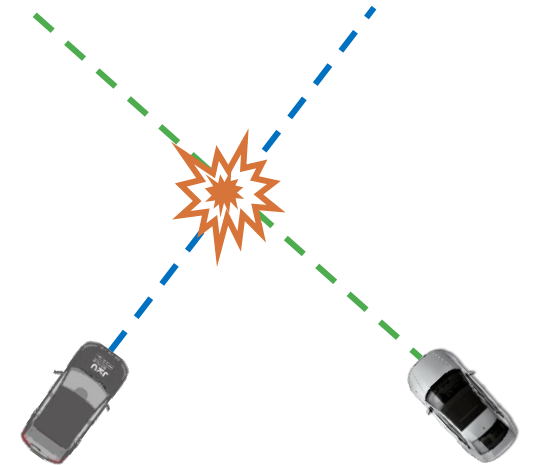


Collision Detected



Collision Detected

Note: even though the object can be moving, but the static collision detection will assume it to be static.

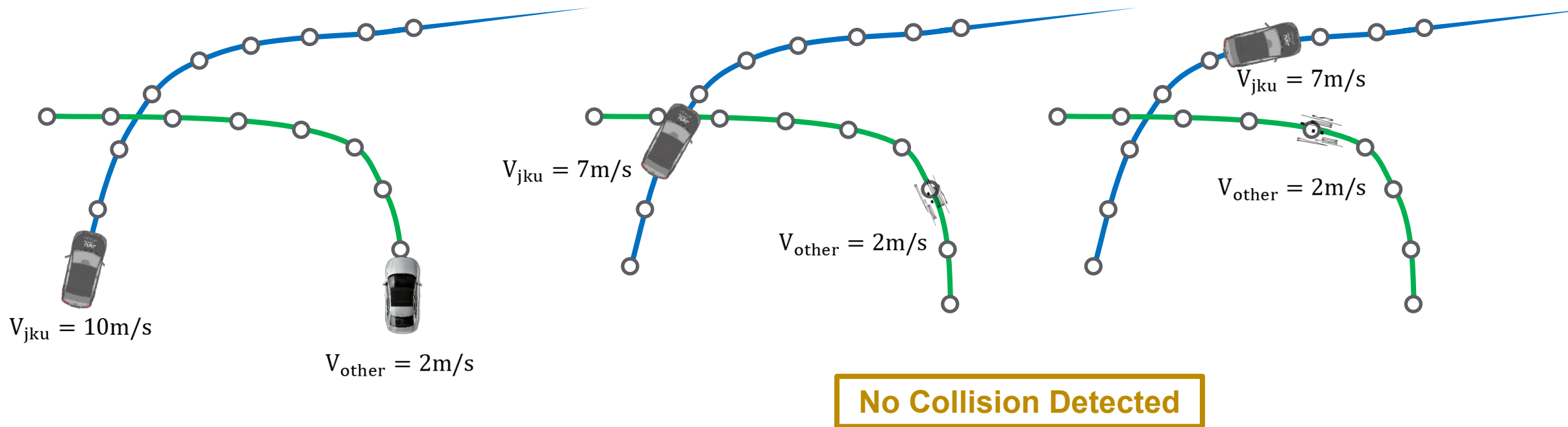


No Collision Detected!

Collision Detection

Dynamic Collision Detection

Requires a **set of predicted trajectories of all relevant participants** on the driving scene. These trajectories predict where and when each participant will be, allowing us to check for future collisions. Note that paths that intersect may not always lead to a collision.



Collision Detection: Challenges and Solutions

Main Challenges of Collision Detection

We cannot be 100% accurate in our detection and guarantee safety, because we do not have the following:

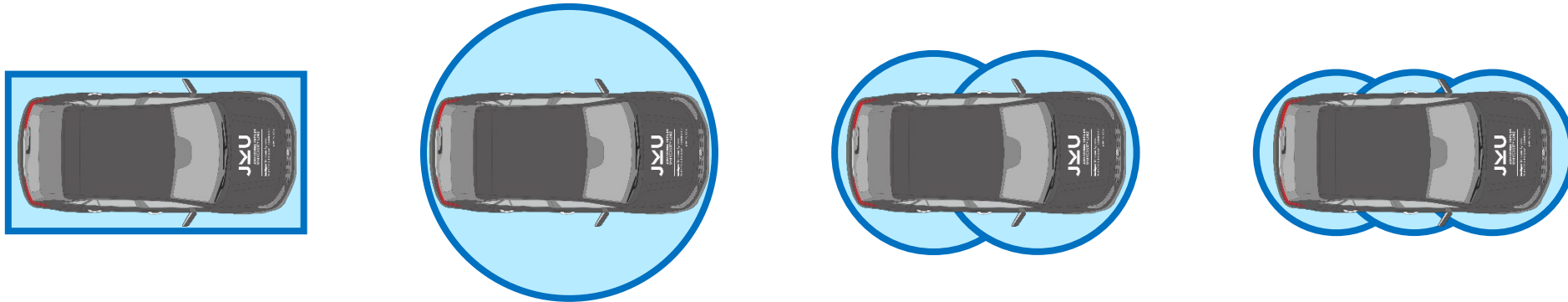
- Perfect observations of the scene. E.g. where everyone is, the state of the road, the weather, etc.
- Perfect internal measurements. These measurements should tell us accurately where we are, what speed and acceleration we're traveling at, etc.
- “Long” computational time, which is not available in real-time applications.
- Highly accurate predictions and intent predictions, which are extremely challenging and an open area of research.

Collision Detection: Challenges and Solutions

How should the solution be

- Since information is imperfect, algorithms need to be robust to noisy observations.
- Since computational time is short, algorithms need to use simple and fast approximations of a perfect collision detection without sacrificing safety.

Most Common Technique Used in Motion Planning for AD: Shape-based Conservative Collision Detection



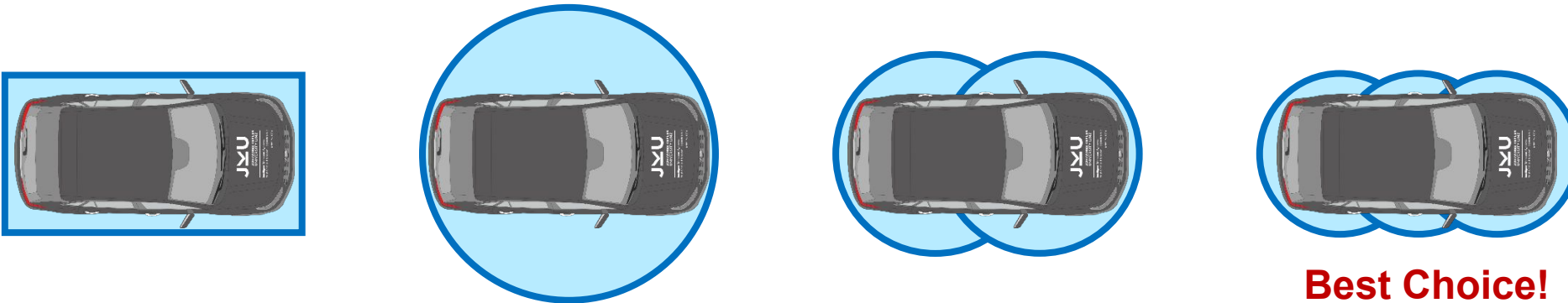
Conservative: The vehicle must be fully enclosed within the shape we use.

Collision Detection: Challenges and Solutions

What Shape Should We Use?

The main objectives to decide what “shape” to use, sizes and location are:

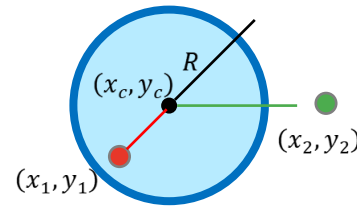
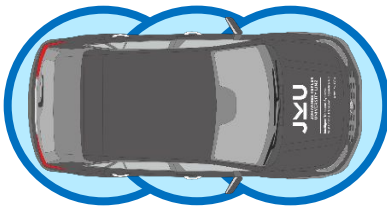
- Allow for a very fast intrusion check in any vehicle orientation - **Circle!**
- Minimize false positive: Reduce the colored areas where there is not a part of vehicle



Circle-Based Collision Detection

Step1: Point mass collision check

Inside or outside the circle: Distance between point (x_i, y_i) and center circle (x_c, y_c)

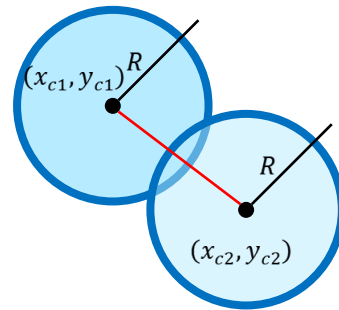
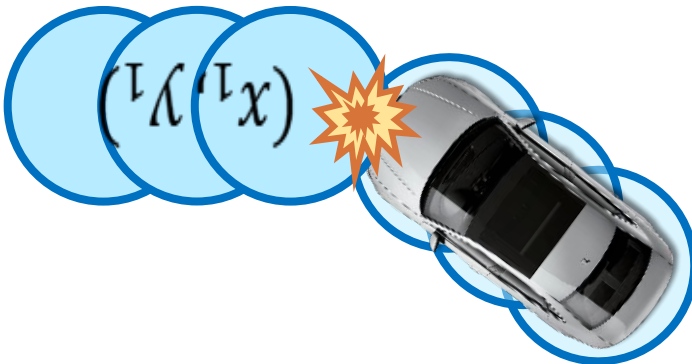


Inside the circle (i.e. collision) if

$$\sqrt{(x_i - x_c)^2 + (y_i - y_c)^2} \leq R$$

Step2: Vehicle to Vehicle collision check

2 circles intersect when the distance between their centers (x_{c1}, y_{c1}) and (x_{c2}, y_{c2}) is smaller than the sum of their radii.



There is a collision if

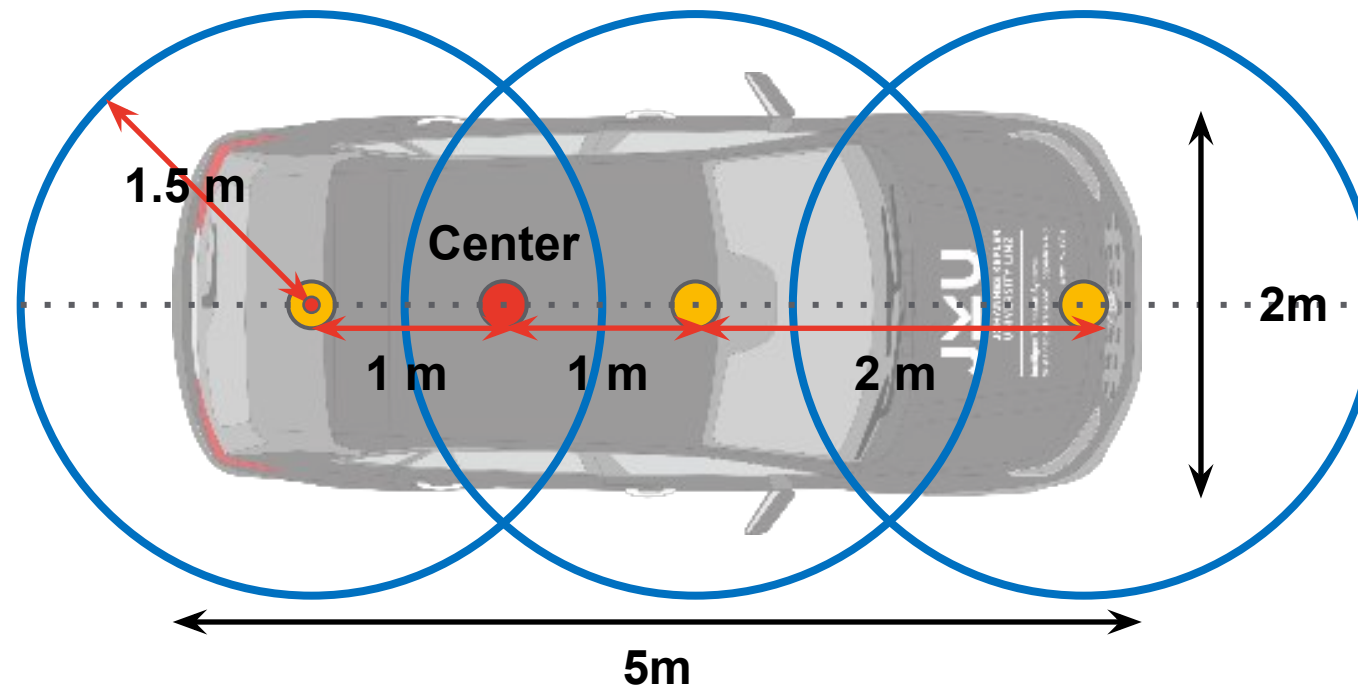
$$\sqrt{(x_{c1} - x_{c2})^2 + (y_{c1} - y_{c2})^2} \leq R_1 + R_2$$

Note: to check for any orientation collision we need to check pairwise all circles, i.e. 9 comparisons

Circle-Based Collision Detection

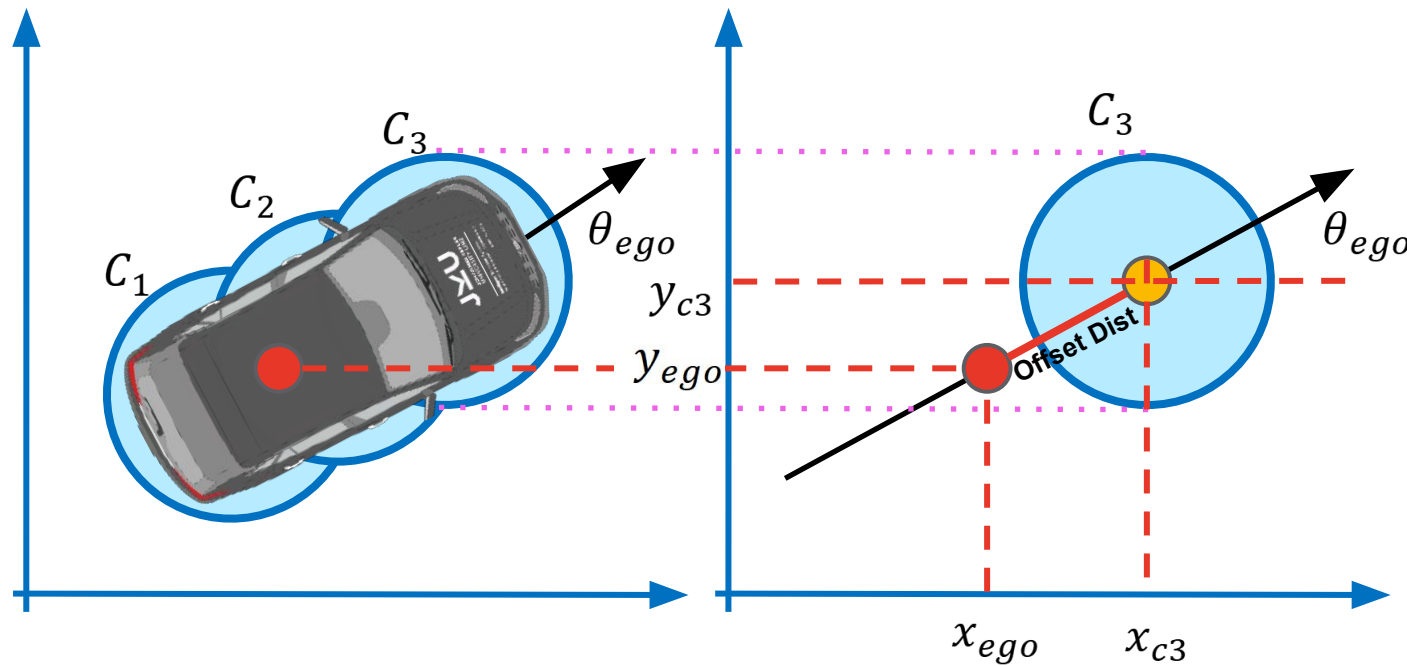
Circles location and size on an average compact car for perfect coverage

- It is safe to say that the **rough average compact car dimensions** are 5m × 2m
- The longitudinal center of the vehicle is reported (CARLA Simulator) to be set at about 2/3 from the front bumper.
- Circles should be located at: **[-1m, +1m, +3m]** On the centerline
- With **1.5 m Radius**



Circle-Based Collision Detection

How do we account for vehicles' heading when placing the circles?
i.e. X and Y coordinates of circles' center



Circle center location

$$x_{c3} = x_{ego} + \text{Offset dist} * \cos(\theta_{ego})$$

$$y_{c3} = y_{ego} + \text{Offset dist} * \sin(\theta_{ego})$$

For C1 and C2 follow the same concept

Exercise: Circle-Based Collision Detection

Problem Statement

In this exercise you will generate a circle-based collision detection function. To do so, you'll have to:

1. Calculate the center coordinates (x,y) of the 3 circles that will represent every car. To do that, you will use:
2. CIRCLE_OFFSETS = distance that the circles should be at with reference to the center of the vehicle and on the center-line of the vehicle.
3. CIRCLE_RADII = the radii of the circles.
4. The vehicles' heading/yaw.

Once the circles are placed (i.e. we know their centers' coordinates) for both, the ego car and the "actor", we can proceed to check for collisions. Here you will have to, pairwise (taking 1 circle from ego and 1 circle from the actor) check if they intersect (collision) or not.

Download code from here: EX Planning-20230423T175220Z-001 in Moodle, part 3
Follow the "TODO"s to complete this exercise.